

```

import { useState, useCallback, useEffect } from "react";

const SIZE = 15, WIN = 5;
const makeBoard = () => Array.from({length:SIZE}, ()=>Array(SIZE).fill(null));

function checkWin(b,r,c) {
  const p=b[r][c]; if(!p) return null;
  for(const [dr,dc] of [[0,1],[1,0],[1,1],[1,-1]]) {
    let n=1;
    for(let i=1;i<WIN;i++){const nr=r+dr*i,nc=c+dc*i;if(nr<0||nr>=SIZE||nc<0||nc>
    for(let i=1;i<WIN;i++){const nr=r-dr*i,nc=c-dc*i;if(nr<0||nr>=SIZE||nc<0||nc>
    if(n>=WIN) return p;
  }
  return null;
}

function checkAnyWin(b) {
  for(let r=0;r<SIZE;r++) for(let c=0;c<SIZE;c++) if(b[r][c]&&checkWin(b,r,c)) re
  return null;
}

function scorePos(b,r,c,pl) {
  if(b[r][c]!==null) return -1;
  const op=pl==="black"? "white": "black"; let tot=0;
  for(const [dr,dc] of [[0,1],[1,0],[1,1],[1,-1]]) {
    let m=1,b1=0;
    for(let i=1;i<=4;i++){const nr=r+dr*i,nc=c+dc*i;if(nr<0||nr>=SIZE||nc<0||nc>=
    for(let i=1;i<=4;i++){const nr=r-dr*i,nc=c-dc*i;if(nr<0||nr>=SIZE||nc<0||nc>=
    if(m>=5)tot+=100000;else if(m===4&&b1===0)tot+=10000;else if(m===4)tot+=1000;
    let o=1,ob=0;
    for(let i=1;i<=4;i++){const nr=r+dr*i,nc=c+dc*i;if(nr<0||nr>=SIZE||nc<0||nc>=
    for(let i=1;i<=4;i++){const nr=r-dr*i,nc=c-dc*i;if(nr<0||nr>=SIZE||nc<0||nc>=
    if(o>=5)tot+=90000;else if(o===4&&ob===0)tot+=9000;else if(o===4)tot+=800;els
  }
  return tot;
}

function findBestMove(b,pl) {
  let best=-1,br=-1,bc=-1;
  const ck=new Set();
  for(let r=0;r<SIZE;r++)for(let c=0;c<SIZE;c++)if(b[r][c]!==null)
    for(let dr=-2;dr<=2;dr++)for(let dc=-2;dc<=2;dc++){const nr=r+dr,nc=c+dc;if(n
    if(ck.size===0) return {r:7,c:7};
  for(const k of ck){const[r,c]=k.split(",").map(Number);const s=scorePos(b,r,c,p

```

```

    return {r:br,c:bc};
}

/* Find longest line of a player */
function findLongestLine(b,pl) {
    let bestLen=0,bestCells=[];
    for(let r=0;r<SIZE;r++)for(let c=0;c<SIZE;c++){
        if(b[r][c]!==pl) continue;
        for(const[dr,dc] of [[0,1],[1,0],[1,1],[1,-1]]){
            const cells=[r,c];
            for(let i=1;i<SIZE;i++){const nr=r+dr*i,nc=c+dc*i;if(nr<0||nr>=SIZE||nc<0||nc>=SIZE){break;}
            if(cells.length>bestLen){bestLen=cells.length;bestCells=cells;}
        }
    }
    return {len:bestLen,cells:bestCells};
}

/* =====
SKILL DEFINITIONS
===== */
const SKILLS = [
    { id:"feisha", name:"飞沙走石", emoji:"💨", desc:"扔掉对手的棋子! 选择一颗敌方棋子, 连
    needTarget:true, targetFilter:(b,r,c,me)=>b[r][c]===(me==="black"? "white": "bl
    execute:(b,r,c,me)=>{
        const op=me==="black"? "white": "black", nb=b.map(x=>[...x]); let rm=0;
        for(let dr=-2;dr<=2;dr++)for(let dc=-2;dc<=2;dc++){
            const nr=r+dr,nc=c+dc;
            if(nr>=0&&nr<SIZE&&nc>=0&&nc<SIZE&&nb[nr][nc]===op){nb[nr][nc]=null;rm++;}
        }
        return {board:nb, msg:`💨 飞沙走石! 扔掉了 ${rm} 颗棋子`};
    },
    { id:"jingru", name:"静如止水", emoji:"🧊", desc:"冻结对手, 跳过对方下一回合", color:
    needTarget:false,
    execute:(b)=>({board:b.map(x=>[...x]), msg:"🧊 静如止水! 对手被冻结一回合", skipO
    },
    { id:"liba", name:"力拔山兮", emoji:"💪", desc:"翻转2颗敌方棋子为己方", color:"#F59E
    needTarget:false,
    execute:(b,_r,_c,me)=>{
        const op=me==="black"? "white": "black", nb=b.map(x=>[...x]);
        const es=[]; for(let r=0;r<SIZE;r++)for(let c=0;c<SIZE;c++){if(nb[r][c]===op)es.push([r,c]);}
        const n=Math.min(2,es.length);
        for(let i=0;i<n;i++){const idx=Math.floor(Math.random()*es.length);const[er,ec]=es[idx];
        return {board:nb, msg:`💪 力拔山兮! 翻转了 ${n} 颗棋子`};
    },
    { id:"baojie", name:"保洁上门", emoji:"🧹", desc:"选一列, 整列所有棋子全部清除", colc

```

```

    needTarget:true, targetMode:"column", targetFilter:()=>true,
    execute:(b,_r,c)=>{
      const nb=b.map(x=>[...x]); let rm=0;
      for(let r=0;r<SIZE;r++){if(nb[r][c]!==null)rm++;nb[r][c]=null;}
      return {board:nb, msg:`🧹 保洁上门! 清除了第 ${c+1} 列共 ${rm} 颗棋子`};
    }
  },
  { id:"seeyou", name:"See You Again", emoji:"👋", desc:"手刀! 消除敌方最长的一条连线",
    needTarget:false,
    execute:(b,_r,_c,me)=>{
      const op=me=="black"? "white": "black", nb=b.map(x=>[...x]);
      const {len,cells}=findLongestLine(nb,op);
      cells.forEach(([r,c])=>{nb[r][c]=null;});
      return {board:nb, msg:`👋 See You Again! 消除了 ${len} 颗连线`};
    }
  },
  { id:"dongshan", name:"东山再起", emoji:"🔄", desc:"在棋盘空位随机新增2颗己方棋子", c
    needTarget:false,
    execute:(b,_r,_c,me)=>{
      const nb=b.map(x=>[...x]);
      const emp=[]; for(let r=0;r<SIZE;r++)for(let c=0;c<SIZE;c++)if(nb[r][c]==n
      const n=Math.min(2,emp.length);
      for(let i=0;i<n;i++){const idx=Math.floor(Math.random()*emp.length);const[e
      return {board:nb, msg:`🔄 东山再起! 新增了 ${n} 颗棋子`};
    }
  },
];

/* =====
  AI SKILL DECISION
  ===== */
function aiDecideSkill(board, aiUsed, turnNum) {
  if(turnNum < 6) return null; // too early
  const myLine = findLongestLine(board, "white");
  const oppLine = findLongestLine(board, "black");

  // Emergency: player has 4 in a row → See You Again
  if(oppLine.len >= 4 && !aiUsed["seeyou"]) return "seeyou";

  // Player has 3 open → 飞沙走石
  if(oppLine.len >= 3 && !aiUsed["feisha"] && Math.random() > 0.4) return "feisha

  // Random offensive skills
  if(turnNum > 10 && !aiUsed["liba"] && Math.random() > 0.7) return "liba";
  if(turnNum > 8 && !aiUsed["jingru"] && Math.random() > 0.8) return "jingru";
  if(turnNum > 14 && !aiUsed["dongshan"] && Math.random() > 0.6) return "dongshan
  if(turnNum > 12 && !aiUsed["baojie"] && oppLine.len >= 3 && Math.random() > 0.5

```

```

    return null;
}

function aiExecuteSkill(board, skillId) {
    const skill = SKILLS.find(s=>s.id===skillId);
    if(!skill) return null;

    if(skill.needTarget) {
        if(skillId === "feisha") {
            // Target the opponent's longest line center
            const {cells} = findLongestLine(board, "black");
            if(cells.length > 0) {
                const mid = cells[Math.floor(cells.length/2)];
                return skill.execute(board, mid[0], mid[1], "white");
            }
            // fallback: random black stone
            for(let r=0;r<SIZE;r++) for(let c=0;c<SIZE;c++) if(board[r][c]=== "black") r
        }
        if(skillId === "baojie") {
            // Find column with most black stones
            let bestCol=0, bestCount=0;
            for(let c=0;c<SIZE;c++){let cnt=0;for(let r=0;r<SIZE;r++)if(board[r][c]=== "
            return skill.execute(board,0,bestCol,"white");
        }
    }

    return skill.execute(board,0,0,"white");
}

/* =====
    MAIN COMPONENT
    ===== */
export default function SkillGomoku() {
    const [board,setBoard]=useState(makeBoard);
    const [history,setHistory]=useState([]);
    const [winner,setWinner]=useState(null);
    const [thinking,setThinking]=useState(false);
    const [scores,setScores]=useState({you:0,ai:0});
    const [showResult,setShowResult]=useState(false);
    const [myUsed,setMyUsed]=useState({});
    const [aiUsed,setAiUsed]=useState({});
    const [activeSkill,setActiveSkill]=useState(null);
    const [toast,setToast]=useState("");
    const [turnNum,setTurnNum]=useState(0);
    const [streak,setStreak]=useState(0);
    const [aiSkillMsg,setAiSkillMsg]=useState("");

```

```

const cellSize=Math.floor(Math.min(typeof window!=='undefined'?window.innerWidth:
const pad=cellSize, totalSize=cellSize*(SIZE-1)+pad*2;

const flash=(msg)=>{setToast(msg);setTimeout(()=>setToast(""),2800)};
const flashAi=(msg)=>{setAiSkillMsg(msg);setTimeout(()=>setAiSkillMsg(""),3000)

const handleWin=(b,who)=>{
  setWinner(who);
  if(who==="black"){setScores(s=>({...s,you:s.you+1}));setStreak(s=>s+1);}
  else{setScores(s=>({...s,ai:s.ai+1}));setStreak(0);}
  setTimeout(()=>setShowResult(true),800);
};

const doAiTurn=useCallback((curBoard,skip,curAiUsed,curTurn)=>{
  if(skip){setThinking(false);return;}
  setThinking(true);
  setTimeout(()=>{
    let nb=curBoard.map(x=>[...x]);
    const au=curAiUsed||aiUsed;
    const tn=curTurn||turnNum;

    // AI decides whether to use a skill
    const skillChoice = aiDecideSkill(nb, au, tn);

    if(skillChoice) {
      const result = aiExecuteSkill(nb, skillChoice);
      if(result) {
        nb = result.board;
        setBoard(nb.map(x=>[...x]));
        setAiUsed(u=>({...u,[skillChoice]:true}));
        const sk = SKILLS.find(s=>s.id===skillChoice);
        flashAi(`AI 使用了 ${sk.emoji} ${sk.name}!`);

        const w=checkAnyWin(nb);
        if(w){handleWin(nb,w);setThinking(false);return;}

        if(result.skipOpponent) {
          // AI skipped your turn, AI goes again with normal move
          setTimeout(()=>{
            const mv=findBestMove(nb,"white");
            if(mv.r>=0){nb=nb.map(x=>[...x]);nb[mv.r][mv.c]="white";setBoard(nb)
            const w2=checkAnyWin(nb);
            if(w2) handleWin(nb,w2);
            setThinking(false);
          },600);
          return;

```

```

    }
    setThinking(false);
    return;
  }
}

// Normal AI move
const mv=findBestMove(nb,"white");
if(mv.r>=0){
  nb[mv.r][mv.c]="white";
  setBoard(nb.map(x=>[...x]));
  setHistory(h=>[...h,{r:mv.r,c:mv.c,p:"white"}]);
  const w=checkWin(nb,mv.r,mv.c);
  if(w) handleWin(nb,w);
}
setThinking(false);
},400+Math.random()*500);
},[aiUsed,turnNum]);

const placeStone=useCallback((r,c)=>{
  if(winner||thinking) return;

  // Skill target mode
  if(activeSkill) {
    const skill=SKILLS.find(s=>s.id===activeSkill);
    if(skill?.needTarget) {
      if(skill.targetMode==="column") {
        const result=skill.execute(board,r,c,"black");
        const nb=result.board;
        setBoard(nb);setMyUsed(u=>({...u,[skill.id]:true}));
        setActiveSkill(null);flash(result.msg);setTurnNum(t=>t+1);
        const w=checkAnyWin(nb);if(w){handleWin(nb,w);return;}
        doAiTurn(nb,result.skipOpponent,aiUsed,turnNum+1);return;
      }
      if(skill.targetFilter&&!skill.targetFilter(board,r,c,"black")){flash("⚠ 技能目标无效");return;}
      const result=skill.execute(board,r,c,"black");
      const nb=result.board;
      setBoard(nb);setMyUsed(u=>({...u,[skill.id]:true}));
      setActiveSkill(null);flash(result.msg);setTurnNum(t=>t+1);
      const w=checkAnyWin(nb);if(w){handleWin(nb,w);return;}
      doAiTurn(nb,result.skipOpponent,aiUsed,turnNum+1);return;
    }
  }
}

// Normal placement
if(board[r][c]) return;
const nb=board.map(x=>[...x]);

```

```

    nb[r][c]="black";
    setBoard(nb);setHistory(h=>[...h,{r,c,p:"black"}]);setTurnNum(t=>t+1);
    const w=checkWin(nb,r,c);
    if(w){handleWin(nb,w);return;}
    doAiTurn(nb,false,aiUsed,turnNum+1);
  },[board,history,winner,thinking,activeSkill,aiUsed,turnNum,doAiTurn]);

const useSkill=(id)=>{
  if(winner||thinking||myUsed[id]) return;
  const skill=SKILLS.find(s=>s.id===id);
  if(!skill) return;
  if(skill.needTarget){setActiveSkill(id);flash(`🎯 ${skill.name}: ${skill.desc}`);
  const result=skill.execute(board,0,0,"black");
  setBoard(result.board);setMyUsed(u=>({...u,[id]:true}));
  flash(result.msg);setTurnNum(t=>t+1);
  const w=checkAnyWin(result.board);if(w){handleWin(result.board,w);return;}
  doAiTurn(result.board,result.skipOpponent,aiUsed,turnNum+1);
};

const restart=()=>{setBoard(makeBoard());setHistory([]);setWinner(null);setThin

const last=history.length>0?history[history.length-1]:null;
const aiUsedCount=Object.keys(aiUsed).length;

return(<>
  <style>{`
    *{margin:0;padding:0;box-sizing:border-box;-webkit-tap-highlight-color:tran
    body{background:#1A1510;font-family:'Noto Sans SC',sans-serif}
    @import url('https://fonts.googleapis.com/css2?family=Noto+Sans+SC:wght@400
    @keyframes dropIn{from{transform:scale(0);opacity:0}to{transform:scale(1);o
    @keyframes pulse{0%,100%{box-shadow:0 0 0 0 rgba(255,200,60,0.4)}50%{box-sh
    @keyframes winGlow{0%,100%{opacity:1}50%{opacity:0.6}}
    @keyframes overlayIn{from{opacity:0}to{opacity:1}}
    @keyframes cardPop{from{opacity:0;transform:scale(0.85) translateY(20px)}to
    @keyframes bounce{0%{transform:scale(1)}30%{transform:scale(1.3)}50%{transf
    @keyframes aiSkill{from{opacity:0;transform:translateY(-10px)}to{opacity:1;
  `}</style>

  <div style={{minHeight:'100vh',display:'flex',flexDirection:'column',alignIte

    /* Toast - player */
    <div style={{position:'fixed',top:16,left:'50%',transform:`translateX(-50%)

    /* Toast - AI skill */
    <div style={{position:'fixed',top:56,left:'50%',transform:`translateX(-50%)

    /* Header */

```

```

<div style={{textAlign:'center',marginBottom:6}}>
  <h1 style={{fontSize:20,fontWeight:900,color:'#E8C96B',letterSpacing:2}}>
    <p style={{fontSize:10,color:'#8A7A60',marginTop:2}}>加入技能，好好玩，要爆了
  </div>

  /* Score */
  <div style={{display:'flex',gap:20,marginBottom:6,fontSize:12}}>
    <div style={{display:'flex',alignItems:'center',gap:5}}>
      <div style={{width:10,height:10,borderRadius:'50%',background:'#1A1510'}>
        <span>你 <b style={{color:'#E8C96B'}}>{scores.you}</b></span>
      </div>
      <div style={{fontSize:10,color:'#555'}}>vs</div>
      <div style={{display:'flex',alignItems:'center',gap:5}}>
        <div style={{width:10,height:10,borderRadius:'50%',background:'#E8E0D0'}>
          <span>AI <b style={{color:'#E8C96B'}}>{scores.ai}</b></span>
        </div>
      </div>
    </div>

    /* Status */
    <div style={{marginBottom:6,fontSize:11,height:16,color:activeSkill?'#E8C96'
      {activeSkill ? `🎯 请选择目标 · 点击棋盘` : winner==="black"?`🏆 你赢了! ":wi
    </div>

    /* Board */
    <div style={{width:totalSize,height:totalSize,position:'relative',backgroun
      <svg style={{position:'absolute',inset:0}} width={totalSize} height={tota
        <Array.from({length:SIZE}, (_,i)=><g key={i}>
          <line x1={pad} y1={pad+i*cellSize} x2={pad+(SIZE-1)*cellSize} y2={pad
          <line x1={pad+i*cellSize} y1={pad} x2={pad+i*cellSize} y2={pad+(SIZE-
        </g>)}
        {[[3,3],[3,11],[7,7],[11,3],[11,11]].map(([r,c])=><circle key={`-${r}-${
      </svg>

      <Array.from({length:SIZE}, (_,r)=>Array.from({length:SIZE}, (_,c)=>
        <div key={`ck-${r}-${c}`} onClick={()=>placeStone(r,c)}
          style={{position:'absolute',left:pad+c*cellSize-cellSize/2,top:pad+r*
        ))}

      {board.map((row,r)=>row.map((cell,c)=>{
        if(!cell) return null;
        const isLast=last&&last.r===r&&last.c===c;
        const isB=cell==="black";
        return(<div key={`s-${r}-${c}`} style={{
          position:'absolute',left:pad+c*cellSize-cellSize*0.4,top:pad+r*cellSi
          width:cellSize*0.8,height:cellSize*0.8,borderRadius:'50%',
          background:isB?'radial-gradient(circle at 35% 35%,#555,#111 60%,#000)
          boxShadow:isB?'1px 2px 4px rgba(0,0,0,0.6)':'1px 2px 4px rgba(0,0,0,0

```

```

        zIndex:3,animation:'dropIn 0.2s ease',
        ...(isLast&&!winner?{animation:'dropIn 0.2s ease, pulse 1.5s ease inf
        ...(winner?{opacity:winner===cell?1:0.4,animation:winner===cell?'winG
    }}>
    {isLast&&!winner&&<div style={{position:'absolute',inset:0,borderRadi
</div>;
  }}}
</div>

{/* YOUR SKILLS */}
<div style={{width:totalSize,marginTop:10}}>
  <div style={{fontSize:10,color:'#6A5A40',marginBottom:4,display:'flex',ju
    <span>⚡ 你的技能</span>
    {activeSkill&&<span style={{color:'#E8C96B',cursor:'pointer',fontSize:1
</div>
  <div style={{display:'flex',gap:4,flexWrap:'wrap'}}>
    {SKILLS.map(sk=>{
      const used=myUsed[sk.id];
      const active=activeSkill===sk.id;
      return(<button key={sk.id} onClick={()=>useSkill(sk.id)} disabled={us
        style={{
          flex:'1 1 auto',minWidth:56,padding:'6px 4px',borderRadius:6,
          background:active?'rgba(232,201,107,0.15)':used? '#1A1510': '#2A221
          border:`1.5px solid ${active? '#E8C96B':used? '#2A2218': '#3A3020'}`
          color:used? '#3A3020':sk.color,fontSize:10,fontFamily:'inherit',
          cursor:used||winner||thinking?'default':'pointer',textAlign:'cent
          opacity:used?0.3:1,display:'flex',flexDirection:'column',alignIte
        }}>
        <span style={{fontSize:14}}>{sk.emoji}</span>
        <span style={{fontWeight:600,fontSize:9}}>{sk.name}</span>
      </button>);
    })}
  </div>
</div>

{/* AI SKILLS STATUS */}
<div style={{width:totalSize,marginTop:6}}>
  <div style={{fontSize:10,color:'#6A5A40',marginBottom:4}}>🤖 AI 技能</div>
  <div style={{display:'flex',gap:4,flexWrap:'wrap'}}>
    {SKILLS.map(sk=>{
      const used=aiUsed[sk.id];
      return(<div key={sk.id} style={{
        flex:'1 1 auto',minWidth:56,padding:'4px',borderRadius:6,
        background:used?'rgba(239,68,68,0.08)': '#1A1510',
        border:`1px solid ${used?'rgba(239,68,68,0.2)': '#2A2218'}`,
        textAlign:'center',fontSize:9,color:used? '#EF4444': '#3A3020',
        display:'flex',flexDirection:'column',alignItems:'center',gap:1,

```

```

    >
    <span style={{fontSize:12}}>{sk.emoji}</span>
    <span>{used?'已使用':'未使用'}</span>
  </div>;
  })
</div>
</div>

{/* Buttons */}
<div style={{display:'flex',gap:10,marginTop:12}}>
  <button onClick={restart} style={{
    padding:'9px 22px',borderRadius:8,
    background:winner?'linear-gradient(135deg,#C8A96B,#B8944A)':'#3A3020',
    border:winner?'none':'1px solid #4A4030',color:winner?'#1A1510':'#C8A96',
    fontSize:13,fontWeight:winner?700:400,fontFamily:'inherit',cursor:'pointer'
  }}>{winner?'再来一局':'重新开始'}</button>
</div>

<div style={{marginTop:6,fontSize:10,color:'#555'}}>
  第 {turnNum} 回合 · 你用了 {Object.keys(myUsed).length} 个技能 · AI用了 {aiUsed} 个技能
</div>

{/* RESULT */}
{showResult&&<div style={{position:'fixed',inset:0,zIndex:100;background:'white'>
  <div style={{background:'linear-gradient(160deg,#2A2218,#1A1510,#2A2218)'>
    <div style={{fontSize:44,marginBottom:14,animation:winner==='black'?'fadeOut':null}}>
      <div style={{fontSize:26,fontWeight:800,color:winner==='black'?'#E8C96B':'white'}}>
        <div style={{fontSize:12,color:'#8A7A60',marginBottom:18}}>{winner==='black'?<div style={{display:'flex',justifyContent:'center',gap:24,marginBottom:24}}>
          <div><div style={{fontSize:9,color:'#6A5A40',marginBottom:2}}><div><div style={{fontSize:9,color:'#6A5A40',marginBottom:2}}>你的技能</div><div style={{fontSize:9,color:'#6A5A40',marginBottom:2}}>AI技能</div><div style={{fontSize:9,color:'#6A5A40',marginBottom:2}}>总胜</div>
        </div>
      </div>
    </div>
  </div>
  <div style={{display:'flex',gap:8,justifyContent:'center'}}>
    <button onClick={()=>setShowResult(false)} style={{padding:'9px 16px'}}>
    <button onClick={restart} style={{padding:'9px 20px',borderRadius:8,border:1px solid #4A4030'}}>
  </div>
</div>
</div>
</div>
</div>
</div>
}

```